Compound queries

A `bool` query is a compound query with several clauses (`must`, `must_not`, `should`, and `filter`) to wrap leaf queries. The `must_not` clause contradicts the `must` query's intent. Both `filter` and `must_not` clauses are executed in a `filter` context. We can also use the `filter` context in a bool query, we can also use it in a `constant_score` (the name is a giveaway) query.

The `constant_score` query is another compound query in which we can attach a query in a `filter` clause. The following query shows this in action:

```
GET movies/_search
{
  "query": {
    "constant_score": {
      "filter": {
        "match": {
          "title": "Godfather"
        }
      }
    }
  }
}
```

We look at compound queries in chapter 11.

Knowing the execution context gets us one step closer to understanding the inner workings of the Elasticsearch engine. The filter execution context helps create performant queries because the additional effort of running the relevancy algorithm is not required. We look at examples that demonstrate these contexts in the following chapters.

Now that we've had an overview of search in general (and with search data at hand), let's look at what constitutes a search request and dissect the results (search response).

8.5 Anatomy of a request and a response

We had a quick glance at the search request and response in the last few chapters without worrying about attribute details and explanations. However, it is important to understand the request and response objects, to formulate the query object without errors and understand the meaning of the attributes featured in the response. In this section, we delve into the request and response objects.

8.5.1 Search requests

Search queries can be executed using a URI request method or Query DSL. As we discussed earlier, we focus on the Query DSL in this book as it is more powerful and expressive. Figure 8.2 shows the anatomy of a search request.